ANNAMALAI UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E. INFORMATION TECHNOLOGY

VII-SEMESTER

22ITEP706 – IoT AND ITS APPLICATIONS LABORATORY

LABORATORY RECORD

(July 2026 – November 2026)

Name: _____

Register No.: _____

ANNAMALAI UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E. INFORMATION TECHNOLOGY

VII-SEMESTER

22ITEP706 – IoT AND ITS APPLICATIONS LABORATORY

Certified that this is the Bonafide record of work done by Mr./Ms._____

Reg.No. _____of B.E. (Information Technology) in the **22ITEP706 – IoT AND ITS APPLICATION LABORATORY** during the odd semester of the academic year 2026– 2027.

Place: Annamalai Nagar

Date: / /2026

Staff in-charge

Internal Examiner **External Examiner**

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E. INFORMATION TECHNOLOGY

VISION:

To produce globally competent, quality technocrats, to inculcate values of leadership and research qualities and to play a vital role in the socio-economic progress of the nation.

MISSION:

- To partner with the University community to understand the information technology needs of faculty, staff and students.
- To develop dynamic IT professionals with globally competitive learning experience by providing high class education.
- To involve graduates in understanding need-based Research activities and disseminate the knowledge to develop entrepreneur skills.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO	PEO Statements
PEO1	To offer students with **core competence** in mathematical, scientific and basic engineering rudiments necessary to prepare, analyze and solve hardware/software engineering problems and/or also to pursue advanced study or research.
PEO2	To educate students with good **scope** of knowledge in core areas of IT and related engineering so as to comprehend engineering trade-offs, analyze, design, and synthesize data and technical concepts to create novel products and solutions for the real-life problems.
PEO3	To instil in students to maintain high **proficiency** and ethical standards, effective oral and written communication skills, to work as part of teams on multidisciplinary projects and diverse professional environments, and relate engineering issues to the society, global economy and to emerging technologies.
PEO4	To deliver our graduates with **learning environment** awareness of the life-long learning needed for a successful professional career and to introduce them to written ethical codes and guidelines, perform excellence, leadership and demonstrate good citizenship.

COURSE OBJECTIVES:

- To know the different real time sensors used to measure the different electrical parameters.
- To control the different electrical devices from anywhere through IoT.
- To understand the architectural features of Embedded systems for IoT applications.

LIST OF EXERCISES:

1. Study of IoT protocols
2. Blink Inbuilt LED on ESP32
3. Control Servo Motor using ESP32
4. Buzzer Control using ESP32
5. (a)Ultrasonic Sensor with LED Indicator
   (b)Ultrasonic Sensor with Buzzer Indicator
6. Sending Email using ESP32
7. DHT Sensor Data Upload to ThingSpeak
8. Wi-Fi Based Home Automation (Without Relay)

COURSE OUTCOMES:

At the end of the course, the students will be able to

1. Synthesize new IOT protocols by doing some minor changes.

2. Apply the different architectural features of Embedded systems for IOT application design.

3. Apply the concepts for sensors and IOT for health care applications.

Mapping of Course Outcomes (COs) with Programme Outcomes (POs) and Program Specific Outcomes (PSOs)															
	PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PO12**	**PSO1**	**PSO2**	**PSO3**
CO1	3	3	2	2	-	3	2	2	2	-	2	2	3	3	2
CO2	3	3	2	2	-	2	3	2	2	-	3	2	3	3	2
CO3	2	2	3	2	-	2	2	2	2	-	2	3	2	2	2

CONTENTS

S.No.	Date	NAME OF THE EXPERIMENT	Page No.	Signature
1		Study of IoT protocols		
2		Blink Inbuilt LED on ESP32		
3		Control Servo Motor using ESP32		
4		Buzzer Control using ESP32		
5		(a)Ultrasonic Sensor with LED Indicator		
		(b)Ultrasonic Sensor with Buzzer Indicator		
6		Sending Email using ESP32		
7		DHT Sensor Data Upload to ThingSpeak		
8		Wi-Fi Based Home Automation (Without Relay)		

Ex.No.: 01 Date:	Study of IoT protocols

AIM:

To understand the basic IoT Protocols.

DESCRIPTION:

IoT protocols are the communication standards that enable devices to connect and exchange data within the Internet of Things ecosystem. These protocols ensure that devices can communicate with each other, regardless of their manufacturer or platform. They are crucial for the successful implementation of IoT projects, as they govern how data is transmitted and received.

TYPES OF IOT PROTOCOLS:

IoT protocols can be broadly categorized into:

- **Network Layer Protocols:**These protocols manage the communication and connectivity between devices in the IoT system.
- **Wi-Fi:** Enables wireless communication over local area networks.
- **Bluetooth:** A short-range wireless technology for personal area networks.
- **Zigbee:** A low-power, low-data rate wireless technology often used in home automation and sensor networks.
- **Cellular (4G/5G):** Provides long-range, high-bandwidth connectivity for IoT devices.
- **LPWANs (Low-Power Wide-Area Networks):** Technologies like Sigfox and LoRaWAN are designed for long-range, low-power communication.
- **Data/Application Layer Protocols:**These protocols handle the actual data exchange and communication between devices.
- **MQTT (Message Queuing Telemetry Transport):** A lightweight messaging protocol based on the publish-subscribe model, widely used for

machine-to-machine communication.

- **CoAP (Constrained Application Protocol):** Designed for constrained devices and networks, using a request/response model.
- **HTTP (Hypertext Transfer Protocol):** The foundation of the World Wide Web, also used in some IoT applications.
- **DDS (Data Distribution Service):** A middleware protocol for real-time, reliable data exchange, often used in industrial IoT.
- **AMQP (Advanced Message Queuing Protocol):** A more complex messaging protocol suitable for reliable, enterprise-level IoT deployments.
- **XMPP (Extensible Messaging and Presence Protocol):** A protocol for real-time communication, used in various applications, including IoT.

KEY CONSIDERATIONS WHEN CHOOSING PROTOCOLS:

- **Range:** The distance over which devices need to communicate.
- **Bandwidth:** The amount of data that needs to be transmitted.
- **Power Consumption:** The energy requirements of the devices.
- **Security:** The need for encryption, authentication, and other security measures.
- **Complexity:** The overhead and complexity of implementing the protocol.

RESULT:

Thus the protocols of IoT were studied carefully.

Ex.No.: 02	Blink Inbuilt LED on ESP32
Date:	

AIM:

To understand digital output control by blinking the onboard LED at a 1-second interval using Arduino IDE and SimulIDE.

INTRODUCTION:

The onboard LED is used as a simple output indicator. Blinking it helps beginners understand GPIO pin operations, timing, and microcontroller programming basics. In this experiment the circuit is simulated in SimulIDE, so no physical hardware is required.

WORKING PRINCIPLE:

- The digital GPIO pin outputs a HIGH or LOW signal to turn the LED ON or OFF.
- pinMode() configures the pin as an output.
- The delay() function controls the blink interval (1000 ms = 1 second).

SOFTWARE TOOLS REQUIRED:

- Arduino IDE (to write and compile the program and export the .hex file)
- SimulIDE (to simulate the circuit and load the compiled firmware)

HARDWARE TOOLS REQUIRED:

- No physical components. The microcontroller board and its onboard LED are virtual components placed in SimulIDE.

DESCRIPTION:

A microcontroller board is a low-cost programmable device with digital input/output (I/O) pins, analog-to-digital converters (ADC), pulse-width modulation (PWM) and serial communication interfaces, which makes it suitable for IoT, automation and embedded system projects. Programs are written in the Arduino IDE using simple functions such as pinMode(), digitalWrite() and delay().

The board has an onboard LED connected to digital pin 13, which is often used as a

simple output device for basic experiments. SimulIDE is a real-time electronic circuit simulator that allows the compiled firmware to be loaded on a virtual microcontroller, so the behaviour of the circuit can be observed without any physical hardware.

PIN CONFIGURATION:

- Digital pin 13 → Onboard LED

PROCEDURE:

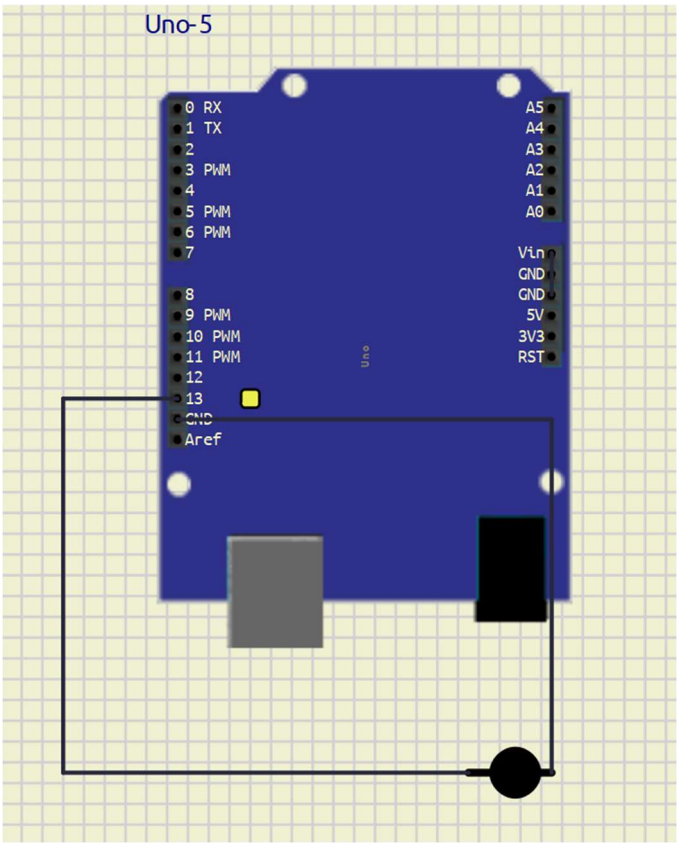
1. Open Arduino IDE and write the program.
2. Select the board and compile the program (Verify). Export the compiled binary (Sketch → Export Compiled Binary) to get the .hex file.
3. Open SimulIDE and place the microcontroller board on the circuit area.
4. Right-click the microcontroller → Load Firmware and select the .hex file.
5. Click the Power/Play button to start the simulation.
6. Observe the LED turning ON for 1 second and OFF for 1 second.

PROGRAM:

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

OUTPUT:



RESULT:

The onboard LED successfully blinked at 1-second intervals in the simulation, demonstrating digital output control.

Ex.No.: 03	Control Servo Motor using ESP32
Date:	

AIM:

To rotate a servo motor to different angular positions (20°, 90° and 160°) using PWM signals generated by the microcontroller, simulated using Arduino IDE and SimulIDE.

INTRODUCTION:

Servo motors are widely used in robotics for precise angular motion. The microcontroller generates PWM signals to control the servo position. In this experiment the servo is a virtual component in SimulIDE.

WORKING PRINCIPLE:

- The PWM pulse width determines the servo angle: 1 ms → 0°, 1.5 ms → 90°, 2 ms → 180°.
- The Servo library (Servo.h) simplifies PWM signal generation.
- myservo.write(angle) moves the servo shaft to the given angle, and delay() holds it there for the specified time.

SOFTWARE TOOLS REQUIRED:

- Arduino IDE
- Servo library (Servo.h, included with Arduino IDE)
- SimulIDE

HARDWARE TOOLS REQUIRED:

- No physical components. The microcontroller board and the servo motor are virtual components placed in SimulIDE.

DESCRIPTION:

A servo motor is a rotary actuator that allows precise control of angular position, velocity and acceleration. It consists of a DC motor, a gear system, a control circuit and a position sensor (potentiometer). Servo motors are widely used in robotics, automation systems and IoT projects where accurate positioning is required.

In this experiment, the microcontroller controls the position of a servo motor by generating a Pulse Width Modulated (PWM) signal. The shaft position depends on the pulse width of the signal received.

The PWM signal sent to the servo typically has:

- A frequency of 50 Hz (20 ms period)
- A pulse width between 1 ms and 2 ms, corresponding to a 0° to 180° rotation of the servo motor

In the program, the servo is attached to pin 13 and moved to 20° (hold 500 ms), then 90° (hold 500 ms), then 160° (hold 1000 ms). This sequence repeats continuously.

Key Features of Servo Motor:

- Precise control of angular position
- Operates on low voltage (typically 5V)
- Requires a PWM control signal
- Commonly used in robotic arms, pan-tilt cameras and automation systems

PIN CONFIGURATION:

- Digital pin 13 → Servo Signal
- 5V → Servo VCC
- GND → Servo GND

PROCEDURE:

1. Open Arduino IDE and write the program (include the Servo library).
2. Compile the program and export the compiled binary (.hex file).
3. Open SimulIDE and place the microcontroller board and a servo motor.
4. Connect the servo signal pin to pin 13, VCC to 5V and GND to ground.
5. Right-click the microcontroller → Load Firmware and select the .hex file.
6. Start the simulation and observe the servo moving to 20°, 90° and 160° repeatedly.

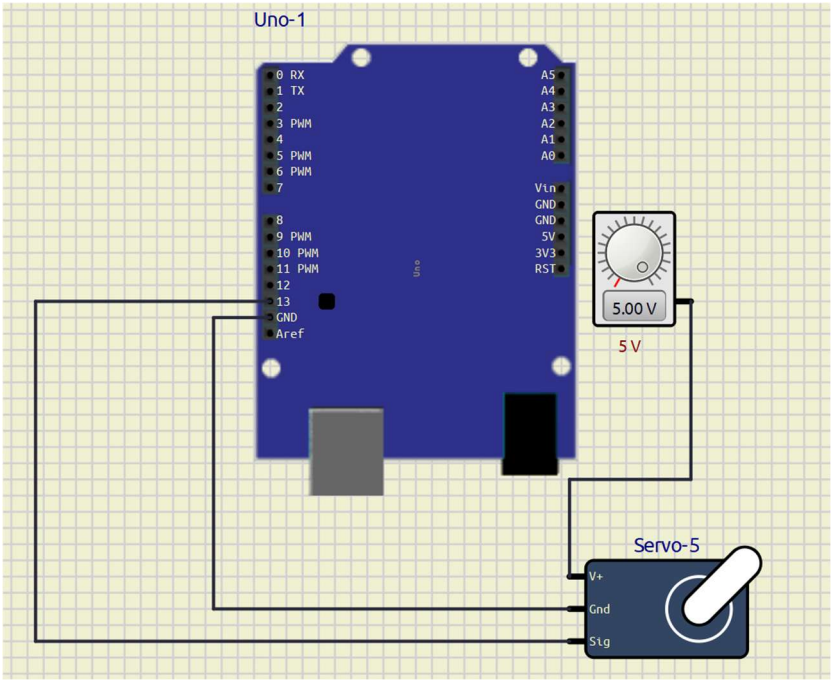
PROGRAM:

```
#include<Servo.h>
 Servo myservo;
```

```
 int servoPin=13;
void setup() {
  myservo.attach(servoPin);
}

void loop() {
    myservo.write(20);
    delay(500);
    myservo.write(90);
    delay(500);
     myservo.write(160);
    delay(1000);
}
```

OUTPUT:



RESULT:

Servo control was successful in the simulation, validating PWM output and motor interfacing.

Ex.No.: 04	Buzzer Control using ESP32
Date:	

AIM:

To generate an audible alert by turning a buzzer ON and OFF at 1-second intervals using Arduino IDE and SimulIDE.

INTRODUCTION:

Buzzers are simple output devices used for alerts. Controlling a buzzer helps in understanding GPIO digital output in practical scenarios. Here the buzzer is a virtual component in SimulIDE.

WORKING PRINCIPLE:

- Applying HIGH to the buzzer pin produces sound.
- Applying LOW turns the sound off.
- delay() controls the ON/OFF duration.

SOFTWARE TOOLS REQUIRED:

- Arduino IDE
- SimulIDE

HARDWARE TOOLS REQUIRED:

- No physical components. The microcontroller board and the buzzer are virtual components placed in SimulIDE.

DESCRIPTION:

A buzzer is an electronic device that produces sound when an electrical signal is applied. It is widely used in electronic and IoT applications as an audio alert system, for example in alarms, timers and notifications.

The microcontroller can control external devices through its GPIO (General Purpose Input/Output) pins. By connecting a buzzer to one of the GPIO pins, it can be turned ON

or OFF by sending HIGH or LOW digital signals.

There are two main types of buzzers:

1. **Active Buzzer:** Produces sound automatically when voltage is applied.
2. **Passive Buzzer:** Requires a frequency signal (PWM) to generate sound.

In this experiment, an active buzzer is used for simplicity. When the microcontroller sends a HIGH signal the buzzer sounds; when it sends a LOW signal it stops. This experiment shows how microcontrollers interact with hardware devices to give real-world responses based on program logic.

PIN CONFIGURATION:

- Digital pin 12 → Buzzer (+)
- GND → Buzzer (−)

PROCEDURE:

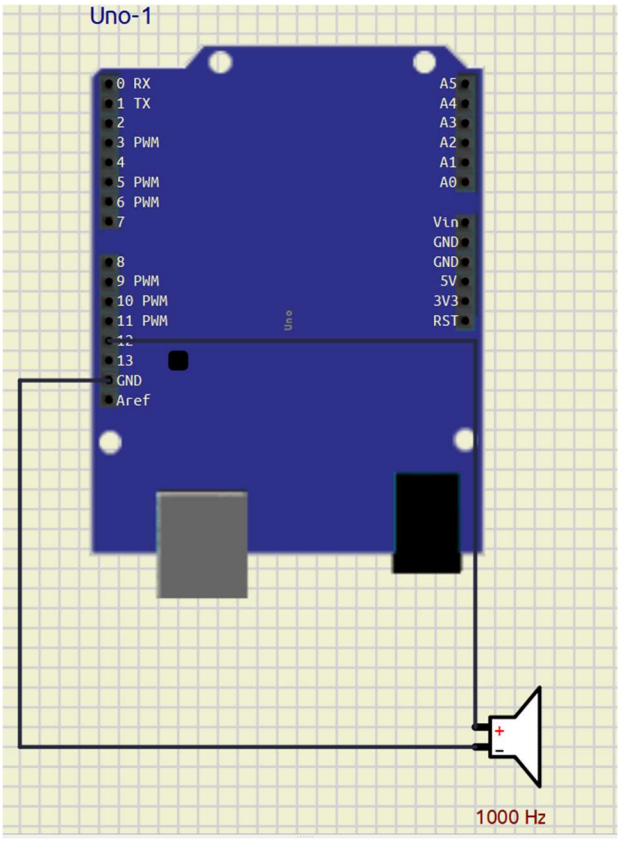
1. Open Arduino IDE and write the program.
2. Compile the program and export the compiled binary (.hex file).
3. Open SimulIDE, place the microcontroller board and a buzzer.
4. Connect the buzzer (+) to pin 12 and the buzzer (−) to ground.
5. Right-click the microcontroller → Load Firmware and select the .hex file.
6. Start the simulation and observe the buzzer turning ON and OFF every second.

PROGRAM:

```
void setup() {
  pinMode(12, OUTPUT);
}

void loop() {
  digitalWrite(12, HIGH);  // Turn buzzer ON
  delay(1000);             // Wait for 1 second
  digitalWrite(12, LOW);   // Turn buzzer OFF
  delay(1000);             // Wait for 1 second
}
```

OUTPUT:



RESULT:

The buzzer successfully generated audible signals in the simulation, demonstrating GPIO digital output control.

Ex.No.: 05(a)	Ultrasonic Sensor with LED Indicator
Date:	

AIM:

To measure the distance of an object using the HC-SR04 ultrasonic sensor and indicate proximity with an LED, using Arduino IDE and SimulIDE.

INTRODUCTION:

Ultrasonic sensors measure distance by emitting sound waves and calculating the time taken for them to reflect from an object. An LED provides a simple visual alert for proximity detection.

WORKING PRINCIPLE:

- The TRIG pin sends ultrasonic pulses.
- The ECHO pin receives the reflected pulse.
- Distance = (Time × Speed of Sound) / 2
- The LED turns ON if the object is closer than 10 cm.

SOFTWARE TOOLS REQUIRED:

- Arduino IDE
- SimulIDE (Serial Monitor/Serial Port component to view readings)

HARDWARE TOOLS REQUIRED:

- No physical components. The microcontroller board, HC-SR04 sensor, LED (with series resistor) and Serial Monitor are virtual components in SimulIDE.

DESCRIPTION:

The Ultrasonic Sensor (HC-SR04) measures the distance of an object using ultrasonic sound waves. It works on the principle of echo, the same way bats and dolphins detect objects.

Working Principle:

- The sensor emits high-frequency sound waves (typically 40 kHz) through its transmitter when triggered by the Trig pin.
- When these waves hit an object, they bounce back toward the sensor.
- The receiver (Echo pin) detects the reflected sound waves.
- The time taken for the waves to travel to the object and return is measured.

Using this time, the distance to the object is calculated with the formula:

Distance = Time × Speed of Sound / 2

In the program, the speed of sound is taken as 0.0343 cm/µs, so distance (cm) = (duration × 0.0343) / 2.

Name of the pin	Function
VCC	Power supply, connect to 5V
Trig	Trigger input, sends a short pulse to trigger ultrasonic sound waves
Echo	Echo output, receives reflected waves and outputs a pulse
GND	Ground, connects to system ground

- The ultrasonic sensor measures the distance between the sensor and an obstacle.
- If the object is within the set distance (less than 10 cm), the LED connected to the microcontroller turns ON.
- If the object is beyond the set distance, the LED remains OFF.
- This can be used for proximity detection, automatic lighting or obstacle alert systems.

PIN CONFIGURATION:

- Digital pin 5 → TRIG
- Digital pin 13 → ECHO

- Digital pin 2 → LED (through a resistor)
- 5V → HC-SR04 VCC
- GND → HC-SR04 GND and LED cathode

PROCEDURE

1. Open Arduino IDE and write the program.
2. Compile the program and export the compiled binary (.hex file).
3. Open SimulIDE, place the microcontroller board, HC-SR04 sensor, LED with resistor and a Serial Monitor.
4. Connect TRIG to pin 5, ECHO to pin 13, the LED to pin 2, and power/ground as per the pin configuration.
5. Right-click the microcontroller → Load Firmware and select the .hex file.
6. Start the simulation and change the distance value of the HC-SR04 in SimulIDE.
7. Observe the distance readings in the Serial Monitor (9600 baud) and the LED behaviour.

PROGRAM:

```
#define TRIG_PIN 5
#define ECHO_PIN 13
#define LED_PIN 2

void setup() {
  Serial.begin(9600);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(2, OUTPUT);
}

void loop() {
  // Send ultrasonic pulse
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
```

```
  // Measure echo duration
  long duration = pulseIn(ECHO_PIN, HIGH);

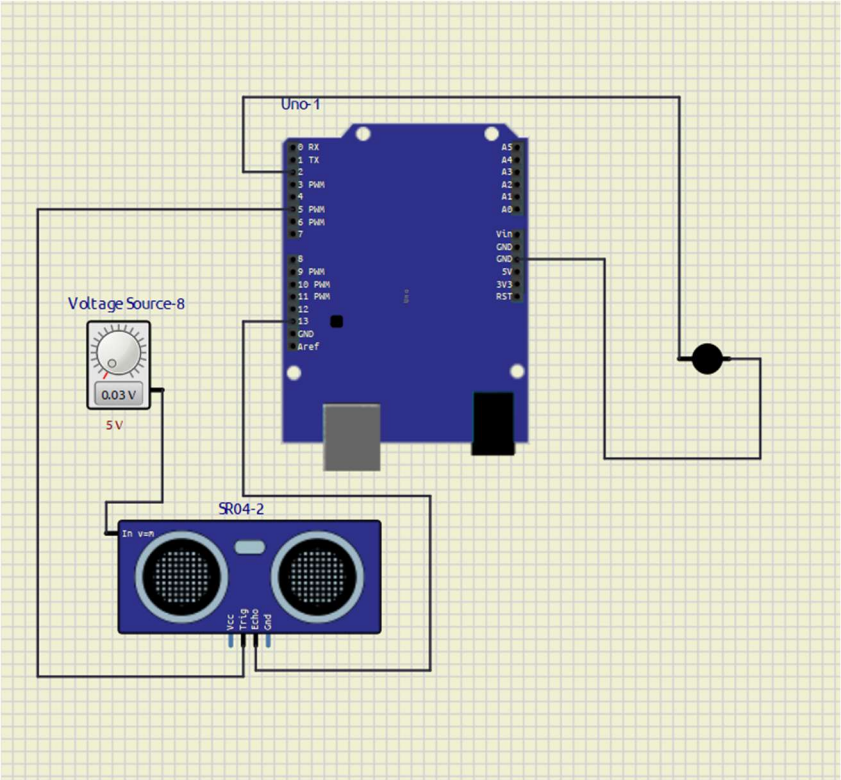
  // Calculate distance in centimeters
  float distance = (duration * 0.0343) / 2;

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  // Turn LED ON if object is closer than 10 cm
  if (distance > 0 && distance < 10) {
    digitalWrite(2 , HIGH);
  } else {
    digitalWrite(2, LOW);
  }

  delay(300);
}
```

OUTPUT:



RESULT:

Ultrasonic distance measurement with LED indicator was successfully implemented in the simulation.

Ex.No.: 05(b)	Ultrasonic Sensor with Buzzer Indicator
Date:	

AIM:

To measure the distance of an object using the HC-SR04 ultrasonic sensor and trigger a buzzer if the object is within a certain range, using Arduino IDE and SimulIDE.

INTRODUCTION:

Ultrasonic sensors measure distance by sending high-frequency sound waves and calculating the time taken for the echo to return. This experiment interfaces the microcontroller with the ultrasonic sensor and activates a buzzer if an object is detected close to the sensor.

WORKING PRINCIPLE:

1. The microcontroller sends a 10-microsecond pulse to the trigger pin of the ultrasonic sensor.
2. The sensor emits ultrasonic waves and waits for the echo.
3. The echo pin goes HIGH when the waves return, and the duration is measured (with a 30 ms timeout).
4. If no echo is received, "No echo detected" is printed and the buzzer stays OFF.
5. Otherwise, distance is calculated using the formula: Distance = Time × Speed of Sound / 2
6. The buzzer turns ON when the distance is less than 10 cm.

SOFTWARE TOOLS REQUIRED:

- Arduino IDE
- SimulIDE (Serial Monitor/Serial Port component to view readings)

HARDWARE TOOLS REQUIRED:

- No physical components. The microcontroller board, HC-SR04 sensor, buzzer and Serial Monitor are virtual components in SimulIDE.

DESCRIPTION:

The Ultrasonic Sensor with Buzzer Indicator system detects the distance between an obstacle and the sensor using ultrasonic waves, and alerts the user through a buzzer when the object is within a certain range. This system is commonly used in obstacle detection, parking assistance and distance measurement applications.

The ultrasonic sensor (HC-SR04) emits ultrasonic sound waves when the trigger pin is pulsed, and the waves reflect back upon hitting an object. The echo pin outputs a pulse whose width equals the travel time of the wave, and the microcontroller measures this time using pulseIn(). Using this time, the distance is determined from the speed of sound.

When the measured distance falls below the threshold (10 cm), the buzzer connected to the microcontroller is activated, producing an audible alert. A timeout of 30000 μs is used in pulseIn(), so if no object is detected within range the program reports "No echo detected" and keeps the buzzer OFF.

Key Features:

- Detects objects without physical contact.
- Provides an audible warning through a buzzer.
- Handles the no-echo case using a timeout.
- Useful in automation, robotics and safety systems.

PIN CONFIGURATION:

- Digital pin 5 → TRIG (HC-SR04)
- Digital pin 13 → ECHO (HC-SR04)
- Digital pin 2 → Buzzer (+)
- GND → Buzzer (−), HC-SR04 GND
- 5V → HC-SR04 VCC

PROCEDURE:

1. Open Arduino IDE and write the program.
2. Compile the program and export the compiled binary (.hex file).
3. Open SimulIDE, place the microcontroller board, HC-SR04 sensor, buzzer and a Serial Monitor.
4. Connect the components as per the pin configuration.

5. Right-click the microcontroller → Load Firmware and select the .hex file.
6. Start the simulation and change the distance value of the HC-SR04.
7. View the distance readings in the Serial Monitor (9600 baud) and observe the buzzer when the object is within 10 cm.

PROGRAM:

```
#define TRIG_PIN 5
#define ECHO_PIN 13
#define BUZZER_PIN 2

void setup() {
  Serial.begin(9600);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(2, OUTPUT);

  digitalWrite(TRIG_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
  // Send ultrasonic trigger pulse
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Measure echo
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);

  // Check if echo was received
  if (duration == 0) {
    Serial.println("No echo detected");
    digitalWrite(BUZZER_PIN, LOW);
    delay(500);
    return;
  }
```

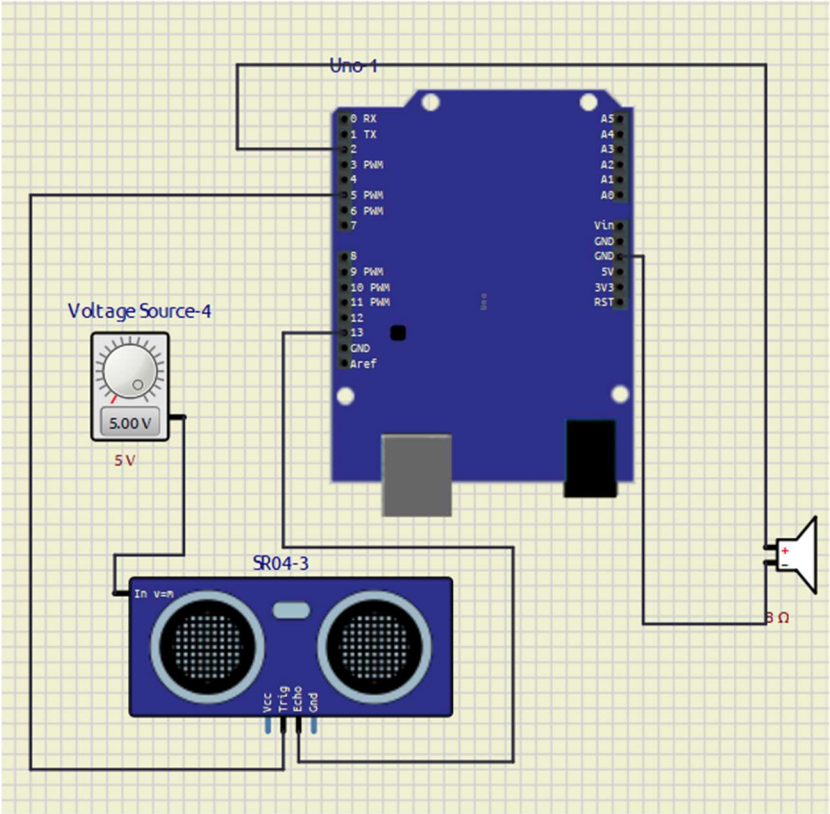
```
  // Calculate distance
  float distance = (duration * 0.0343) / 2;

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  // Turn buzzer ON when object is closer than 10 cm
  if (distance > 0 && distance < 10) {
    digitalWrite(2, HIGH);
  }
  else {
    digitalWrite(2, LOW);
  }

  delay(300);
}
```

OUTPUT:



RESULT:

Ultrasonic distance measurement with buzzer indicator was successfully implemented in the simulation.

Ex.No.:06	Sending Email using ESP32
Date:	

AIM:

To send an automated email using the ESP32 over Wi-Fi via the Gmail SMTP server, simulated using the Wokwi simulator, VS Code and PlatformIO.

INTRODUCTION:

The ESP32 can connect to Wi-Fi and communicate with servers over the internet. Sending an email demonstrates an IoT application in notifications and alerts. In this experiment the ESP32 runs in the Wokwi simulator, which provides a virtual Wi-Fi network, so no physical hardware is required.

WORKING PRINCIPLE:

- The ESP32 connects to the Wokwi virtual Wi-Fi network (Wokwi-GUEST).
- The SMTP client establishes a secure connection with the Gmail server (smtp.gmail.com, port 465).
- The email message is composed (sender, recipient, subject, body) and sent to the recipient.
- The Serial Monitor shows the connection and sending status.

SOFTWARE TOOLS REQUIRED:

- VS Code with the PlatformIO extension
- Wokwi simulator (Wokwi for VS Code)
- ESP Mail Client library (mobizt)
- Gmail account with a Google App Password

HARDWARE TOOLS REQUIRED:

- No physical components. A virtual ESP32 DevKit board is used in Wokwi, with internet access through the simulator.

DESCRIPTION:

The ESP32 is a Wi-Fi and Bluetooth enabled microcontroller widely used in IoT

applications that need internet connectivity. One of the most practical IoT applications is sending automated email alerts based on sensor readings or system events.

In this experiment, the ESP32 is configured to connect to a Wi-Fi network and send an email through an SMTP (Simple Mail Transfer Protocol) server. SMTP is the standard protocol used to send emails over the Internet. By integrating the ESP32 with an SMTP server such as Gmail, it can transmit messages automatically without user intervention.

For example, the ESP32 can be programmed to send an alert email when:

- A temperature sensor exceeds a threshold,
- A motion sensor detects movement, or
- A system process completes successfully.

The program uses the ESP Mail Client library, which communicates with the SMTP server over SSL/TLS. Once connected, the ESP32 composes the message and sends it. The email is sent only once, from setup(), and loop() is left empty. A Google App Password (16 characters, requires 2-step verification) is used instead of the normal Gmail password.

Key Features:

- Uses the SMTP protocol for sending emails
- Requires Wi-Fi connectivity (virtual Wi-Fi in Wokwi)
- Supports secure authentication (SSL/TLS)
- Reports Wi-Fi, SMTP and sending status on the Serial Monitor
- Demonstrates integration of IoT with cloud communication

PIN CONFIGURATION:

No external components are connected. Data flow: ESP32 → Wi-Fi (Wokwi-GUEST) → Gmail SMTP → Email Recipient

PROCEDURE:

1. Create a new PlatformIO project in VS Code (Board: Espressif ESP32 Dev Module, Framework: Arduino).
2. Add the ESP Mail Client library in platformio.ini under lib_deps.

3. Enable 2-step verification for the Gmail account and generate a 16-character App Password (Google Account → Security → App passwords).
4. In main.cpp, set AUTHOR_EMAIL (sender Gmail ID), AUTHOR_PASSWORD (App Password) and RECIPIENT_EMAIL (receiver's email).
5. Create the wokwi.toml and diagram.json files as shown below.
6. Build the project (PlatformIO: Build) so that firmware.bin and firmware.elf are generated.
7. Open diagram.json and start the Wokwi simulation.
8. Open the Serial Monitor (115200 baud) and observe the Wi-Fi connected, SMTP connected and Email sent successfully messages.
9. Check the recipient's inbox for the email.

PROGRAM:

main.cpp

```
#include <WiFi.h>
#include <ESP_Mail_Client.h>

// Wokwi virtual Wi-Fi
#define WIFI_SSID "Wokwi-GUEST"
#define WIFI_PASSWORD ""

// Gmail SMTP settings
#define SMTP_HOST "smtp.gmail.com"
#define SMTP_PORT 465

// Gmail credentials
// Use your Gmail address and a NEW Google App Password
#define AUTHOR_EMAIL "type sender’s e-mail"
#define AUTHOR_PASSWORD "16 letter password"

// Recipient email
#define RECIPIENT_EMAIL "type receiver’s e-mail"

SMTPSession smtp;

void setup()
{
  Serial.begin(115200);
```

```
delay(1000);

Serial.println();
Serial.println("ESP32 Gmail Mail Sender");

// Connect to Wi-Fi
Serial.println("Connecting to Wi-Fi...");

WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

unsigned long startTime = millis();

while (WiFi.status() != WL_CONNECTED &&
      millis() - startTime < 20000)
{
  delay(500);
  Serial.print(".");
}

Serial.println();

if (WiFi.status() != WL_CONNECTED)
{
  Serial.println("Wi-Fi connection failed!");
  return;
}

Serial.println("Wi-Fi connected!");
Serial.print("IP address: ");
Serial.println(WiFi.localIP());

// Configure SMTP session
ESP_Mail_Session session;

session.server.host_name = SMTP_HOST;
session.server.port = SMTP_PORT;
session.login.email = AUTHOR_EMAIL;
session.login.password = AUTHOR_PASSWORD;
session.login.user_domain = "";
```

```
// Create email
SMTP_Message message;

message.sender.name = "ESP32 Mailer";
message.sender.email = AUTHOR_EMAIL;
message.subject = "ESP32 Test Email";

message.addRecipient("Receiver", RECIPIENT_EMAIL);

message.text.content =
   "Hello! This email was sent from an ESP32 simulation in Wokwi.";

message.text.charSet = "us-ascii";
message.text.transfer_encoding =
   Content_Transfer_Encoding::enc_7bit;

// Connect to Gmail SMTP
Serial.println("Connecting to Gmail SMTP...");

if (!smtp.connect(&session))
{
 Serial.println("SMTP connection failed!");
 Serial.println(smtp.errorReason());
 return;
}

Serial.println("SMTP connected!");

// Send email
Serial.println("Sending email...");

if (!MailClient.sendMail(&smtp, &message))
{
 Serial.println("Email sending failed!");
 Serial.println(smtp.errorReason());
}
else
{
 Serial.println("Email sent successfully!");
}
```

```
  smtp.closeSession();
}

void loop()
{
  // Email is sent only once
}
```

Wokwi.toml

```
[wokwi]
version = 1
firmware = '.pio/build/esp32dev/firmware.bin'
elf = '.pio/build/esp32dev/firmware.elf'
```

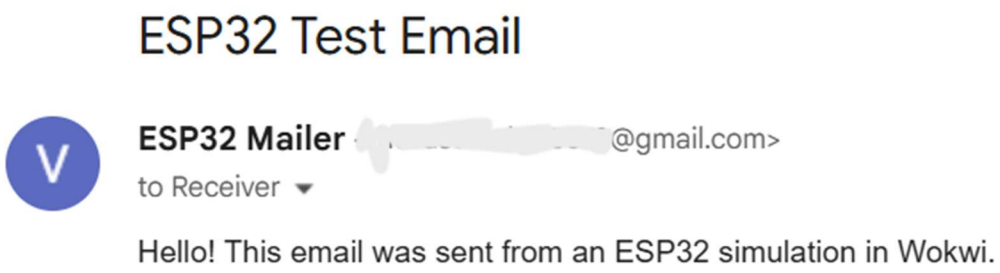
platformio.ini

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
    mobizt/ESP Mail Client
```

diagram.json

```
{
  "version": 1,
  "author": "ESP32 Gmail Sender",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp",
      "top": 0,
      "left": 0,
      "attrs": {}
    }
  ],
  "connections": [],
  "dependencies": {}
}
```

OUTPUT:



RESULT:

The ESP32 successfully sent an email over SMTP in the Wokwi simulation, demonstrating IoT-based communication.

Ex.No.:07	DHT Sensor Data Upload to ThingSpeak
Date:	

AIM:

To measure temperature and humidity using a DHT22 sensor and upload the data to the ThingSpeak cloud, simulated using the Wokwi simulator, VS Code and PlatformIO.

INTRODUCTION:

The DHT22 is a digital sensor that measures temperature and humidity. ThingSpeak is an IoT cloud platform for storing and visualizing sensor data. This experiment demonstrates IoT data acquisition and cloud integration, with the ESP32 and the sensor simulated in Wokwi.

WORKING PRINCIPLE:

- The DHT22 sensor measures temperature and humidity.
- The ESP32 reads the sensor values on GPIO 15 using the DHT library.
- The ESP32 connects to the Wokwi virtual Wi-Fi network (Wokwi-GUEST).
- Data is sent to ThingSpeak over HTTPS using an HTTP GET request containing the Write API Key, Field 1 (temperature) and Field 2 (humidity).
- ThingSpeak returns an entry ID (a number greater than 0) when the upload is accepted.

SOFTWARE TOOLS REQUIRED:

- VS Code with the PlatformIO extension
- Wokwi simulator (Wokwi extension for VS Code)
- Libraries: DHT sensor library (Adafruit) and Adafruit Unified Sensor
- ThingSpeak account

HARDWARE TOOLS REQUIRED:

- No physical components. The ESP32 DevKit board and the DHT22 sensor are virtual parts in Wokwi.
- Internet access on the computer.

DESCRIPTION:

The DHT22 sensor is a low-cost digital sensor used to measure temperature and humidity. It provides calibrated digital output signals, making it ideal for environmental monitoring applications such as weather stations, greenhouse monitoring and IoT-based smart systems.

The DHT22 consists of a capacitive humidity sensor and a thermistor (temperature sensor) along with a small chip that processes the data and sends it as a digital signal to the connected microcontroller, such as the ESP32. It transmits data through a single-wire communication protocol, so only one GPIO pin is needed.

Typical specifications of the DHT22 include:

- **Operating Voltage:** 3.3V to 5V
- **Temperature Range:** −40°C to 80°C (about ±0.5°C accuracy)
- **Humidity Range:** 0% to 100% RH (about ±2–5% accuracy)
- **Sampling Rate:** Once every 2 seconds

In this experiment, the DHT22 sensor in Wokwi is set to a temperature of 24°C and a humidity of 40%. These values can be changed by clicking the sensor during the simulation. The ESP32 reads the values and uploads them to ThingSpeak for visualization. ThingSpeak free accounts accept one update every 15 seconds or more, so the program waits 20 seconds between uploads. The program also checks for failed sensor readings (NaN), reconnects to Wi-Fi if it is lost and validates the response from ThingSpeak.

PIN CONFIGURATION:

DHT22 Pin	ESP32 Pin
VCC	3V3
SDA (DATA)	GPIO 15
GND	GND

- Data flow: ESP32 → Wi-Fi (Wokwi-GUEST) → ThingSpeak Cloud

PROCEDURE:

1. Create a ThingSpeak account and a new channel (Channels → My Channels → New Channel). Enable Field 1 for Temperature and Field 2 for Humidity, then save the channel.
2. Copy the Write API Key of the channel from the API Keys tab.
3. Open VS Code and create a new PlatformIO project (board: Espressif ESP32 Dev Module, framework: Arduino).
4. Add the DHT sensor library and Adafruit Unified Sensor under lib_deps in platformio.ini.
5. Write the program in src/main.cpp and replace WRITE_API_KEY with the Write API Key. Create wokwi.toml and diagram.json in the project root.
6. Build the project using PlatformIO: Build.
7. Start the simulation using Wokwi: Start Simulator (press F1 and choose the command).
8. Observe the temperature and humidity values and the "Upload successful!" message in the Serial Monitor.
9. Open the ThingSpeak channel and check the Field 1 (Temperature) and Field 2 (Humidity) charts. Compare them with the Serial Monitor values.

PROGRAM:

main.cpp

```
#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>

// Sensor wiring
#define DHTPIN 15
#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

// Wokwi virtual Wi-Fi
const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";

// ThingSpeak credentials: replace these values
```

```
const char* WRITE_API_KEY = "Type API Key";

const char* THINGSPEAK_URL =
    "https://api.thingspeak.com/update";

DHT* sensor = &dht;

void setup() {
  Serial.begin(115200);
  delay(1000);

  sensor->begin();

  Serial.println();
  Serial.println("DHT22 ThingSpeak Experiment");
  Serial.println("Connecting to Wokwi Wi-Fi...");

  // Channel 6 is used by the Wokwi-GUEST network
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD, 6);

  int attempts = 0;

  while (WiFi.status() != WL_CONNECTED &&
         attempts < 40) {
    delay(500);
    Serial.print(".");
    attempts++;
  }

  Serial.println();

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("Wi-Fi connected!");
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println("Wi-Fi connection timed out.");
    Serial.println("Check the Wokwi network setup.");
  }
}
```

```
void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Wi-Fi disconnected; reconnecting...");
    WiFi.reconnect();

    delay(5000);

    if (WiFi.status() != WL_CONNECTED) {
      return;
    }
  }

  float temperature = sensor->readTemperature();
  float humidity = sensor->readHumidity();

  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("DHT22 read failed. Check the wiring.");
    delay(2000);
    return;
  }

  Serial.println("----------------------------");
  Serial.printf("Temperature: %.2f C\n", temperature);
  Serial.printf("Humidity: %.2f %%\n", humidity);

  WiFiClientSecure client;
  client.setInsecure();  // Learning/demo use only

  HTTPClient http;
  String url = String(THINGSPEAK_URL) +
               "?api_key=" + WRITE_API_KEY +
               "&field1=" + String(temperature, 2) +
               "&field2=" + String(humidity, 2);

  http.setTimeout(15000);

  if (http.begin(client, url)) {
    int httpCode = http.GET();

    if (httpCode > 0) {
      String response = http.getString();
```

```
    response.trim();

    Serial.printf("HTTP status: %d\n", httpCode);
    Serial.println("ThingSpeak entry ID: " + response);

    if (httpCode == HTTP_CODE_OK &&
        response.toInt() > 0) {
      Serial.println("Upload successful!");
    } else {
      Serial.println("Upload was not accepted.");
      Serial.println("Check your API key and update interval.");
    }
   } else {
    Serial.printf("HTTP request error: %s\n",
                  http.errorToString(httpCode).c_str());
   }

   http.end();
  } else {
   Serial.println("Could not start HTTPS request.");
  }

  // ThingSpeak free accounts require 15 seconds
  // or more between channel updates.
  delay(20000);
}
```

Wokwi.toml

```
[wokwi]
version = 1
firmware = ".pio/build/esp32dev/firmware.bin"
elf = ".pio/build/esp32dev/firmware.elf"
```

platformio.ini

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
```

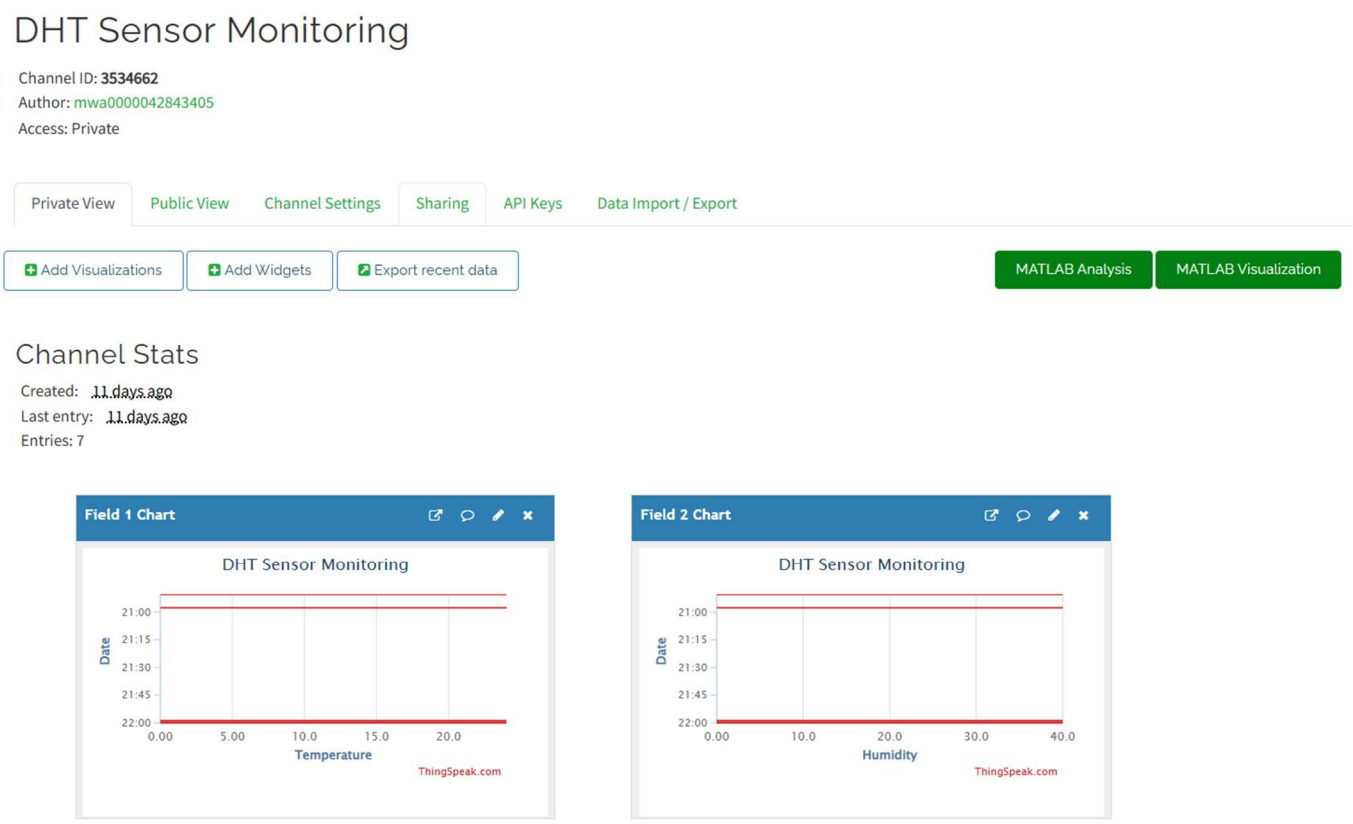
```
lib_deps =
    adafruit/DHT sensor library
    adafruit/Adafruit Unified Sensor
```

diagram.json

```
{
  "version": 1,
  "author": "DHT ThingSpeak Experiment",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-dht22",
      "id": "dht",
      "top": -40,
      "left": 270,
      "attrs": {
        "temperature": "24",
        "humidity": "40"
      }
    }
  ],
  "connections": [
    [
      "esp:3V3",
      "dht:VCC",
      "red",
      []
    ],
    [
      "esp:15",
      "dht:SDA",
      "green",
      []
```

```
    ],
    [
      "esp:GND.1",
      "dht:GND",
      "black",
      []
    ]
  ],
  "dependencies": {}
}
```

OUTPUT:



RESULT:

The ESP32 successfully read temperature and humidity from the DHT22 sensor and uploaded the data to the ThingSpeak cloud in the Wokwi simulation, demonstrating IoT integration.

Ex.No.:08	Wi-Fi Based Home Automation (Without Relay)
Date:	

AIM:

To control four devices (LEDs representing lights, a fan and a socket) using ESP32 over a web interface, simulated using the Wokwi simulator, VS Code and PlatformIO.

INTRODUCTION:

ESP32 can connect to Wi-Fi and serve a web page to control connected devices remotely. This experiment demonstrates a basic home automation system without relays by controlling simulated appliances (LEDs) from a browser.

WORKING PRINCIPLE:

1. The ESP32 connects to the Wokwi virtual Wi-Fi network (Wokwi-GUEST).
2. A web server is started on port 80 using the WebServer library.
3. Wokwi forwards the ESP32's port 80 to localhost:8180 on the computer (net.forward in wokwi.toml), so the page can be opened in a browser.
4. The web page shows a card for each appliance with its status and Turn ON / Turn OFF buttons.
5. When a button is pressed, the browser requests /set?id=<n>&state=<0 or 1>. The ESP32 validates the request, sets the corresponding GPIO pin HIGH or LOW, prints the status on the Serial Monitor and redirects back to the page.

SOFTWARE TOOLS REQUIRED:

- VS Code with the PlatformIO extension
- Wokwi simulator (Wokwi extension for VS Code)
- A web browser

HARDWARE TOOLS REQUIRED:

- No physical components. The ESP32 DevKit board, four LEDs and four 220 Ω resistors are virtual parts in Wokwi.

DESCRIPTION:

The Wi-Fi Based Home Automation System using ESP32 (without relay) is a smart IoT project that allows users to control household appliances such as lights, fans and sockets from a web browser.

This system avoids additional hardware such as relays by connecting low-power devices (LEDs, each with a 220 Ω current-limiting resistor) directly to the ESP32's GPIO pins. The ESP32 acts as a Wi-Fi web server and hosts a web page that displays the status and ON/OFF control buttons for each connected device.

When the user clicks a button on the web page, the ESP32 receives the HTTP GET request, updates the state of the selected appliance, switches the GPIO pin and returns the updated page. The page is styled with HTML and CSS and is responsive, so it works on both computers and mobile screens.

Key Features:

- **Wireless Control:** Operate devices over Wi-Fi using any browser.
- **No Relay Needed:** Direct GPIO control for low-voltage devices (simulated appliances).
- **Status Display:** The page shows whether each appliance is ON or OFF.
- **Input Validation:** Invalid appliance IDs or states are rejected with an error response.
- **Low-Cost Implementation:** ESP32 acts as both controller and web server.

PIN CONFIGURATION:

Appliance	ESP32 Pin	LED Colour
Living Room Light	GPIO 2 → 220 Ω → LED1 → GND	Yellow
Bedroom Light	GPIO 4 → 220 Ω → LED2 → GND	Blue
Fan	GPIO 16 → 220 Ω → LED3 → GND	Green

Appliance	ESP32 Pin	LED Colour
Socket	GPIO 17 → 220 Ω → LED4 → GND	Red

PROCEDURE:

1. Open VS Code and create a new PlatformIO project (board: Espressif ESP32 Dev Module, framework: Arduino).
2. Write the program in src/main.cpp. Create wokwi.toml (with the port forwarding), platformio.ini and diagram.json in the project root.
3. Build the project using PlatformIO: Build.
4. Start the simulation using Wokwi: Start Simulator (press F1 and choose the command).
5. Open the Serial Monitor and wait for "Wi-Fi connected!" and the ESP32 IP address.
6. Open a web browser on the computer and go to http://localhost:8180 (the forwarded address).
7. Click the Turn ON / Turn OFF buttons for each appliance and observe the corresponding LED in the Wokwi circuit and the status message on the page.
8. Record the Serial Monitor messages (for example, "Fan is now ON").

PROGRAM:

main.cpp

```
#include <Arduino.h>
#include <WiFi.h>
#include <WebServer.h>

// Wi-Fi configuration for Wokwi
const char* ssid = "Wokwi-GUEST";
const char* password = "";

// Four simulated appliances
const int appliancePins[] = {2, 4, 16, 17};

const char* applianceNames[] = {
  "Living Room Light",
```

```
  "Bedroom Light",
  "Fan",
  "Socket"
};

const int applianceCount = 4;
bool applianceStates[applianceCount] = {
  false, false, false, false
};

// Web server on port 80
WebServer server(80);

String makePage() {
  String html = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>ESP32 Home Automation</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #eef2f7;
      margin: 0;
      padding: 20px;
      color: #172033;
    }
    .container {
      max-width: 760px;
      margin: auto;
    }
    h1 { text-align: center; }
    .subtitle {
      text-align: center;
      color: #566176;
    }
    .grid {
      display: grid;
```

```
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 16px;
   }
   .card {
    background: white;
    border-radius: 14px;
    padding: 20px;
    box-shadow: 0 3px 12px #0001;
   }
   .icon {
    font-size: 36px;
   }
   .status {
    font-weight: bold;
    margin: 12px 0;
   }
   .on { color: #16803c; }
   .off { color: #b42318; }
   a {
    display: inline-block;
    padding: 11px 18px;
    border-radius: 8px;
    text-decoration: none;
    color: white;
    margin: 4px 3px 0 0;
   }
   .turn-on { background: #16803c; }
   .turn-off { background: #b42318; }
   .footer {
    text-align: center;
    margin-top: 24px;
    color: #566176;
   }
  </style>
</head>
<body>
<div class="container">
  <h1> Smart Home Automation</h1>
  <p class="subtitle">ESP32 Wi-Fi Appliance Control Panel</p>
```

```
  <div class="grid">
)rawliteral";

  for (int i = 0; i < applianceCount; i++) {
    html += "<div class='card'>";
    html += "<h2>" + String(applianceNames[i]) + "</h2>";

    if (applianceStates[i]) {
      html += "<p class='status on'>Status: ON</p>";
    } else {
      html += "<p class='status off'>Status: OFF</p>";
    }

    html += "<a class='turn-on' href='/set?id=";
    html += String(i);
    html += "&state=1'>Turn ON</a>";

    html += "<a class='turn-off' href='/set?id=";
    html += String(i);
    html += "&state=0'>Turn OFF</a>";

    html += "</div>";
  }

  html += R"rawliteral(
  </div>
  <p class="footer">
    Demonstration only: appliances are simulated LEDs.
  </p>
</div>
</body>
</html>
)rawliteral";

  return html;
}

void handleRoot() {
  server.send(200, "text/html", makePage());
```

```
}

void handleSet() {
  if (!server.hasArg("id") || !server.hasArg("state")) {
    server.send(400, "text/plain", "Missing id or state");
    return;
  }

  int id = server.arg("id").toInt();
  int state = server.arg("state").toInt();

  if (id < 0 || id >= applianceCount ||
      (state != 0 && state != 1)) {
    server.send(400, "text/plain", "Invalid appliance or state");
    return;
  }

  applianceStates[id] = (state == 1);
  digitalWrite(appliancePins[id],
               applianceStates[id] ? HIGH : LOW);

  Serial.print(applianceNames[id]);
  Serial.print(" is now ");
  Serial.println(applianceStates[id] ? "ON" : "OFF");

  // Return to the updated control webpage
  server.sendHeader("Location", "/");
  server.send(303, "text/plain", "Appliance state updated");
}

void setup() {
  Serial.begin(115200);
  delay(500);

  for (int i = 0; i < applianceCount; i++) {
    pinMode(appliancePins[i], OUTPUT);
    digitalWrite(appliancePins[i], LOW);
  }
```

```
  Serial.println();
  Serial.println("Wi-Fi Home Automation Starting...");

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password, 6);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi connected!");
  Serial.print("ESP32 IP address: ");
  Serial.println(WiFi.localIP());

  server.on("/", HTTP_GET, handleRoot);
  server.on("/set", HTTP_GET, handleSet);

  server.begin();

  Serial.println("Web server started on port 80");
  Serial.println("Open the forwarded local URL in your browser.");
}

void loop() {
  server.handleClient();
}
```

wokwi.toml

```
[wokwi]
version = 1
firmware = '.pio/build/esp32dev/firmware.bin'
elf = '.pio/build/esp32dev/firmware.elf'

[[net.forward]]
from = "localhost:8180"
to = "target:80"
```

platformio.ini

```
[env:esp32dev]
platform = espressif32@6.10.0
board = esp32dev
framework = arduino
monitor_speed = 115200
```

diagram.json

```
{
  "version": 1,
  "author": "Wi-Fi Home Automation",
  "editor": "wokwi",
  "parts": [
    {
      "type": "board-esp32-devkit-c-v4",
      "id": "esp",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -40,
      "left": 240,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": -40,
      "left": 330,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": 40,
```

```
    "left": 240,
    "attrs": { "value": "220" }
   },
   {
    "type": "wokwi-led",
    "id": "led2",
    "top": 40,
    "left": 330,
    "attrs": { "color": "blue" }
   },
   {
    "type": "wokwi-resistor",
    "id": "r3",
    "top": 120,
    "left": 240,
    "attrs": { "value": "220" }
   },
   {
    "type": "wokwi-led",
    "id": "led3",
    "top": 120,
    "left": 330,
    "attrs": { "color": "green" }
   },
   {
    "type": "wokwi-resistor",
    "id": "r4",
    "top": 200,
    "left": 240,
    "attrs": { "value": "220" }
   },
   {
    "type": "wokwi-led",
    "id": "led4",
    "top": 200,
    "left": 330,
    "attrs": { "color": "red" }
   }
  ],
```

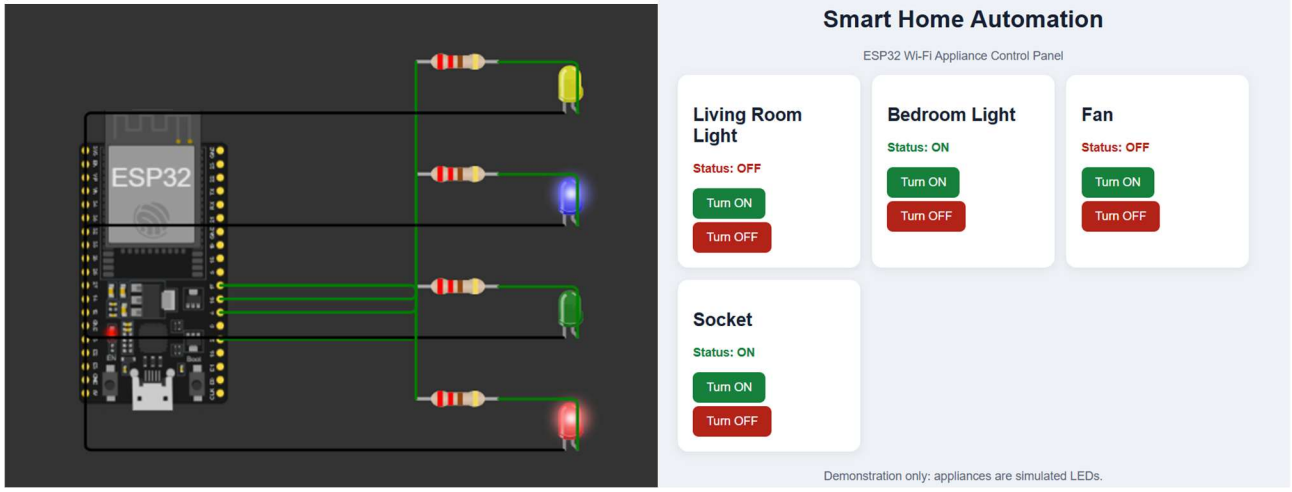
```
  "connections": [
    ["esp:2", "r1:1", "green", []],
    ["r1:2", "led1:A", "green", []],
    ["led1:C", "esp:GND.1", "black", []],

    ["esp:4", "r2:1", "green", []],
    ["r2:2", "led2:A", "green", []],
    ["led2:C", "esp:GND.1", "black", []],

    ["esp:16", "r3:1", "green", []],
    ["r3:2", "led3:A", "green", []],
    ["led3:C", "esp:GND.1", "black", []],

    ["esp:17", "r4:1", "green", []],
    ["r4:2", "led4:A", "green", []],
    ["led4:C", "esp:GND.1", "black", []]
  ],
  "dependencies": {}
}
```

OUTPUT:



RESULT:

A web-based home automation system using ESP32 without relays was successfully implemented and controlled from a browser in the Wokwi simulation.